

DeelMind

FCM Push Notification

Architecture & Flow

Complete engineering reference: iOS ↔ FastAPI ↔ Firebase ↔ Celery ↔
Supabase

2026-05-19 · Internal Engineering Reference

iOS Layer

Backend Layer

Firebase / FCM

Celery Tasks

Beat Tasks

Supabase DB

Confidential — Engineering Use Only

Table of Contents

1. System Architecture Overview

iOS · Backend · FCM · Supabase

2. iOS — Permission & Token Lifecycle

NotifPermManager · AppDelegate · AuthSession

3. Database — Token Storage

Migration · profiles table · fcm_token column

4. Backend — FCM Token Registration

ProfileAPI.swift · PATCH /fcm-token endpoint

5. Backend — Firebase Admin SDK Setup

config.py · notifier.py · _get_firebase_app()

6. Backend — Sending Notifications

send_push_notification() · _Message · APNs config

7. Celery Task — Notification Triggers

tasks.py Step 17 · Step 19 (auto / pending_category)

8. Celery Beat — Timeout Handler

beat_tasks.py · expire_pending_categories()

9. Backend — Category Choice Handler

reels.py PATCH /reels/{id}/category

10. iOS — Notification Response Handling

didReceive · ReelCategoryAPI · CategoriseReelView

11. End-to-End Flow Table

14-step complete notification lifecycle

System Architecture Overview

The ReelMind FCM notification system spans four physical layers: the iOS app, the FastAPI backend (with Celery workers and Beat scheduler), Firebase Cloud Messaging, and the Supabase PostgreSQL database. The diagram below shows all components and how they communicate.

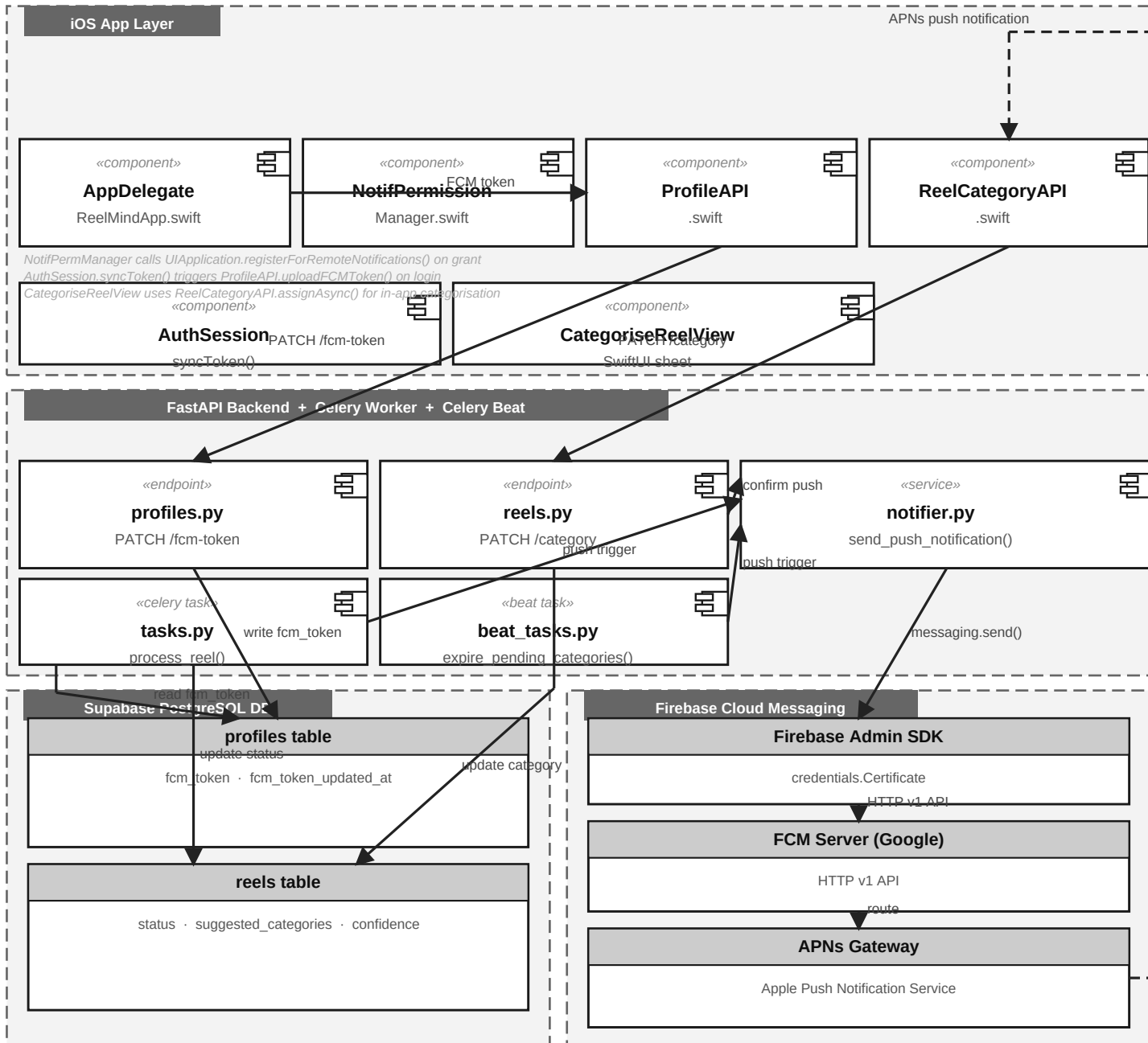


Diagram Legend

■■■■■■■■■■	Solid arrow = direct call / HTTP request / SDK invocation
- - - ■	Dashed arrow = APNs push delivery (external, device-to-device)
[]	White box with component symbol (top-right) = iOS or Backend component
■■■ name	Grey-header box = database table or external service

«**stereotype**»

Italic label in angle brackets = component role / type

Before any FCM push can be delivered to the device, two things must happen: (1) the user must grant notification permission, and (2) the device must register with APNs, which causes Firebase to issue an FCM token. This token is then uploaded to the backend so it can target this device.

2A · NotificationPermissionManager.swift

FILE

`frontend/NotificationPermissionManager.swift`

Manages UNUserNotificationCenter permission state. Observable, drives OnboardingPermissionsView UI.

An **ObservableObject** with a single status property (*notDetermined* / *authorized* / *denied*). Exposes two async methods:

- **refresh()** — reads UNNotificationSettings without prompting. Called on task start in OnboardingPermissionsView.
- **requestOrOpenSettings()** — if *.notDetermined*: calls `requestAuthorization(options: [.alert, .sound, .badge])`. If granted, immediately calls `UIApplication.shared.registerForRemoteNotifications()`. If *.denied*: deep-links to iOS Settings.app.

```
let granted = try await UNUserNotificationCenter.current()
    .requestAuthorization(options: [.alert, .sound, .badge])
if granted {
    UIApplication.shared.registerForRemoteNotifications()
}

// NotificationPermissionManager.requestOrOpenSettings()
```

2B · AppDelegate (ReelMindApp.swift)

FILE

`frontend/ReelMindApp.swift`

UIApplicationDelegate + UNUserNotificationCenterDelegate + MessagingDelegate. Central hub for all FCM and push plumbing.

didFinishLaunchingWithOptions does five things:

- `FirebaseApp.configure()` — boots the Firebase SDK (reads `GoogleService-Info.plist`).
- `Messaging.messaging().delegate = self` — routes token-refresh callbacks here.
- `UNUserNotificationCenter.current().delegate = self` — routes foreground/tap callbacks here.
- Registers the **CATEGORISE** notification category with 4 action buttons (`CAT_0`, `CAT_1`, `CHOOSE_IN_APP`, `UNCATEGORISED`).
- If permission was already granted in a prior session, calls `registerForRemoteNotifications()` so the APNs/FCM token is refreshed.

```
// CATEGORISE category – iOS shows these as notification action buttons
let categoriseActions: [UNNotificationAction] = [
    UNNotificationAction(identifier: 'CAT_0', title: 'Suggestion 1', options: []),
    UNNotificationAction(identifier: 'CAT_1', title: 'Suggestion 2', options: []),
    UNNotificationAction(identifier: 'CHOOSE_IN_APP', title: 'Choose / Create Category',
        options: [.foreground]),
    UNNotificationAction(identifier: 'UNCATEGORISED', title: 'Uncategorised', options: []),
]
let categoriseCategory = UNNotificationCategory(
    identifier: 'CATEGORISE', actions: categoriseActions,
    intentIdentifiers: [], options: [])
UNUserNotificationCenter.current().setNotificationCategories([categoriseCategory])
```

didRegisterForRemoteNotificationsWithDeviceToken

APNs delivers a device token (binary Data). AppDelegate forwards this to the Firebase SDK:

Messaging.messaging().apnsToken = deviceToken. Firebase then internally exchanges this for an FCM registration token.

messaging(_:didReceiveRegistrationToken:) — MessagingDelegate

Called by Firebase SDK whenever a new FCM token is available (first launch, token refresh, or app reinstall). Two actions:

- Cache the token to UserDefaults.standard key 'fcmToken' for later use (e.g. if login hasn't happened yet).
- If the user's auth token already exists in App Group defaults (user was logged in), upload immediately via ProfileAPI.uploadFCMToken(token).

```
func messaging(_ messaging: Messaging, didReceiveRegistrationToken fcmToken: String?) {
    guard let token = fcmToken else { return }
    UserDefaults.standard.set(token, forKey: 'fcmToken') // cache
    let groupDefaults = UserDefaults(suiteName: AppConfig.appGroupID)
    if groupDefaults?.string(forKey: AppConfig.authTokenKey) != nil {
        ProfileAPI.uploadFCMToken(token) // logged in – upload now
    }
}
```

2C · AuthSession.swift

FILE

frontend/AuthSession.swift

Manages Supabase JWT session. Also handles FCM token upload on login/session restore.

AuthSession's **syncToken(_)** is called whenever the Supabase auth state changes (sign-in, sign-out, token refresh). When a JWT is present, it also checks for a cached FCM token and uploads it:

- Mirrors the Supabase JWT to App Group UserDefaults (key: supabaseAuthToken) so the Share Extension and ProfileAPI/ReelCategoryAPI can read it.
- If fcmToken exists in UserDefaults.standard (cached by AppDelegate), calls ProfileAPI.uploadFCMToken(fcmToken). This handles the race where the FCM token arrived before the user logged in.

```
// AuthSession.syncToken
if let token = token {
    defaults.set(token, forKey: AppConfig.authTokenKey)
    if let fcmToken = UserDefaults.standard.string(forKey: 'fcmToken') {
        ProfileAPI.uploadFCMToken(fcmToken) // upload cached token post-login
    }
}
```

The FCM token is stored in the `public.profiles` table — one row per user, one token per device (MVP; multi-device would need a separate devices table).

Migration File

FILE

`supabase/migrations/20260505000001_add_fcm_token_to_profiles.sql`

Adds `fcm_token` and `fcm_token_updated_at` to profiles. Creates a partial index for non-NULL tokens.

```
alter table public.profiles
  add column if not exists fcm_token text,
  add column if not exists fcm_token_updated_at timestamptz;

-- Partial index: only index rows that actually have a token
create index if not exists idx_profiles_fcm_token
  on public.profiles(fcm_token)
  where fcm_token is not null;
```

Schema context: profiles table

Column	Type	Notes
<code>id</code>	<code>uuid</code>	FK → <code>auth.users</code> . PK.
<code>fcm_token</code>	<code>text</code>	FCM registration token. NULL until device registers.
<code>fcm_token_updated_at</code>	<code>timestamptz</code>	Timestamp of last token write. Useful for token staleness checks.

Note: The backend reads this column in `tasks.py`, `beat_tasks.py`, and `api/v1/reels.py` using the **service role key** (bypasses RLS) to fetch the `fcm_token` before calling `send_push_notification()`.

4A · ProfileAPI.swift (iOS → Backend call)

FILE

frontend/ProfileAPI.swift

Fire-and-forget PATCH call to upload the device FCM token. Reads auth token from App Group defaults.

A static enum with a single method `uploadFCMToken(_)`. It reads the Supabase JWT from the App Group shared defaults (so it also works if called from the Share Extension context), builds a PATCH request, and fires it as a background URLSession task. Errors are logged but never surfaced to the UI — the push flow is advisory.

```
let url = AppConfig.backendBaseURL
    .appendingPathComponent('api/v1/profiles/fcm-token')
var request = URLRequest(url: url)
request.httpMethod = 'PATCH'
request.setValue('Bearer \(authToken)', forHTTPHeaderField: 'Authorization')
request.httpBody = try? JSONEncoder().encode(['fcm_token': token])
URLSession.shared.dataTask(with: request) { ... }.resume()
```

4B · FCMTokenRequest schema (schemas/reel.py)

FILE

backend/schemas/reel.py

Pydantic model — validates that the request body contains a non-empty fcm_token string.

```
class FCMTokenRequest(BaseModel):
    """Body for PATCH /api/v1/profiles/fcm-token."""
    fcm_token: str # Pydantic rejects missing/null → 422
```

4C · update_fcm_token endpoint (api/v1/profiles.py)

FILE

backend/api/v1/profiles.py

PATCH /api/v1/profiles/fcm-token — writes token to profiles table. JWT-authenticated.

```
@router.patch('/fcm-token', status_code=status.HTTP_200_OK)
async def update_fcm_token(
    payload: FCMTokenRequest,
    user_id: str = Depends(get_current_user_id), # JWT → user UUID
):
    supabase = get_supabase() # service role key — bypasses RLS
    supabase.table('profiles').update({
        'fcm_token': payload.fcm_token,
        'fcm_token_updated_at': datetime.now(timezone.utc).isoformat(),
    }).eq('id', user_id).execute()
    return {'status': 'ok'}
```

Auth dependency `get_current_user_id` (in `api/deps.py`) calls `supabase.auth.get_user(token)` to verify the JWT and extract the UUID. An invalid token raises HTTP 401 before the handler runs.

5A · config.py — Environment Variable

FILE

backend/config.py

Central config class. FIREBASE_SERVICE_ACCOUNT_JSON is the base64-encoded service account JSON.

```
# Firebase Admin SDK service account (base64-encoded JSON) — Step 22 FCM push
FIREBASE_SERVICE_ACCOUNT_JSON = os.getenv('FIREBASE_SERVICE_ACCOUNT_JSON')

# FCM_SERVER_KEY — legacy field, kept for reference but unused
FCM_SERVER_KEY = os.getenv('FCM_SERVER_KEY')
```

How to set this in production (Render)

1. Download the Firebase service account JSON from Firebase Console → Project Settings → Service Accounts → Generate New Private Key.
2. Base64-encode it: `base64 -i service-account.json | tr -d '\n'`
3. In Render dashboard → reelmind-api service → Environment → add env var FIREBASE_SERVICE_ACCOUNT_JSON with the encoded string.

5B · _get_firebase_app() — Lazy Init (notifier.py)

FILE

backend/services/notifier.py

Lazily initializes Firebase Admin SDK. Returns None (graceful no-op) if config is missing.

Uses a module-level singleton `_firebase_app`. On the first call, it decodes the base64 service account JSON, creates a `credentials.Certificate`, and calls `firebase_admin.initialize_app(cred)`. Subsequent calls return the cached app.

```
_firebase_app: firebase_admin.App | None = None

def _get_firebase_app() -> firebase_admin.App | None:
    global _firebase_app
    if _firebase_app is not None:
        return _firebase_app                # already initialized
    cfg = get_config()
    if not cfg.FIREBASE_SERVICE_ACCOUNT_JSON:
        logger.warning('FCM disabled — no service account JSON')
        return None                        # graceful no-op
    service_account = json.loads(base64.b64decode(cfg.FIREBASE_SERVICE_ACCOUNT_JSON))
    cred = credentials.Certificate(service_account)
    _firebase_app = firebase_admin.initialize_app(cred)
    return _firebase_app
```

6A · send_push_notification() — Public API

FILE

backend/services/notifier.py

Non-fatal push wrapper. Any failure is logged and returns False. Never raises.

Parameter	Type	Purpose
fcm_token	str None	Target device token. None → skip silently.
title	str	Notification title text.
body	str	Notification body / subtitle text.
data	dict[str, str] None	Optional key-value data payload (reel_id, status, suggestions JSON).
category_id	str None	iOS UNNotificationCategory ID. 'CATEGORISE' → shows action buttons.
→ returns	bool	True = sent successfully. False = skipped or any error.

6B · _Message and APNs config dataclasses

Rather than using `firebase_admin.messaging.Message` directly (which is a `MagicMock` in tests), the code defines lightweight `@dataclass` stand-ins (`_Message`, `_APNSConfig`, `_APNSPayload`, `_Aps`). These hold real attribute values so tests can inspect them after `messaging.send()` is mocked.

```
# _Aps carries the iOS category identifier → shows action buttons
@dataclasses.dataclass
class _Aps:
    category: str

@dataclasses.dataclass
class _APNSPayload:
    aps: _Aps

@dataclasses.dataclass
class _APNSConfig:
    payload: _APNSPayload

@dataclasses.dataclass
class _Message:
    notification: object # messaging.Notification(title, body)
    data: dict           # {'reel_id': ..., 'status': ..., 'suggestions': ...}
    apns: Optional[object] # None unless category_id is provided
    token: str           # FCM registration token
```

6C · CATEGORISE notification — what the user sees

When `category_id='CATEGORISE'` is passed to `send_push_notification()`, an `_APNSConfig` object is attached with `aps.category = 'CATEGORISE'`. iOS matches this identifier to the registered `UNNotificationCategory` (set up in `AppDelegate`) and shows the four action buttons below the notification banner.

Button Identifier	Label	Behaviour
-------------------	-------	-----------

CAT_0	Suggestion 1	Calls ReelCategoryAPI.assign(reelId, suggestions[0])
CAT_1	Suggestion 2	Calls ReelCategoryAPI.assign(reelId, suggestions[1])
CHOOSE_IN_APP	Choose / Create Category	Foreground action → app opens CategoriseReelView sheet
UNCATEGORISED	Uncategorised	Calls ReelCategoryAPI.assign(reelId, nil) → status = uncategorised

FILE

backend/workers/tasks.py

process_reel Celery task. Runs the full ingestion pipeline. Three FCM push trigger points.

At the start of `process_reel`, the task fetches the user's FCM token from the profiles table. This is done once and stored in `_fcm_token`. A failure here is non-fatal — the pipeline continues, and notifications are silently skipped if the token is `None`.

```
# FCM token – fetched once at pipeline start (non-critical)
_fcm_token: str | None = None
try:
    _profile = supabase.table('profiles')
        .select('fcm_token')
        .eq('id', reel_data['user_id'])
        .single().execute()
    if _profile.data:
        _fcm_token = _profile.data.get('fcm_token')
except Exception as exc:
    log.warning('could not fetch fcm_token | %s', exc)
```

Trigger Point 1 — Step 17: NoSignalError (no content)

If all three signals (transcript, caption, hashtags) are empty, `build_classification_signal()` raises `NoSignalError`. The pipeline marks the reel **uncategorised** and sends a simple informational push.

```
except NoSignalError:
    supabase.table('reels').update({'status': 'uncategorised'}).eq('id', reel_id).execute()
    send_push_notification(
        fcm_token=_fcm_token,
        title='Reel saved',
        body='We couldn\'t categorise it – no audio or caption found',
        data={'reel_id': reel_id, 'status': 'uncategorised'},
    )
    # no category_id → no action buttons
    return {'reel_id': reel_id, 'status': 'uncategorised'}
```

Trigger Point 2 — Step 19: High confidence → auto-assign

If the Llama classifier returns confidence ≥ 0.70 and the category exists in the DB, the reel is automatically assigned. A simple success push is sent (no action buttons needed).

```
_CONFIDENCE_THRESHOLD = 0.70

if classification.confidence >= _CONFIDENCE_THRESHOLD and resolved_category_id:
    supabase.table('reels').update({
        'category_id': resolved_category_id,
        'confidence': classification.confidence,
        'status': 'ready',
    }).eq('id', reel_id).execute()
    send_push_notification(
        fcm_token=_fcm_token,
        title='Reel saved!',
        body=f'Categorised as {classification.category}',
        data={'reel_id': reel_id, 'status': 'ready'},
    )
    # no category_id → no action buttons
```

Trigger Point 3 — Step 19: Low confidence → pending_category

If confidence < 0.70 (or the top category is not in the DB map), the reel enters **pending_category** status. The push includes up to 3 suggestions as a JSON string in the data payload, and category_id='CATEGORISE' to trigger the action buttons on iOS.

```
suggestions = [classification.category] + classification.alternatives[:2]
supabase.table('reels').update({
  'status': 'pending_category',
  'suggested_categories': suggestions,
  'confidence': classification.confidence,
}).eq('id', reel_id).execute()

send_push_notification(
  fcm_token=_fcm_token,
  title='Help us categorise this reel',
  body='Your reel is saved – which fits best? Ignoring saves it to Uncategorized.',
  data={
    'reel_id': reel_id,
    'suggestions': json.dumps(suggestions), # JSON string – parsed by iOS
  },
  category_id='CATEGORISE', # <-- triggers action buttons on iOS
)
```

FILE

backend/workers/beat_tasks.py

Scheduled Celery Beat task. Runs every 30 minutes. Expires pending_category reels after 1 hour.

When a user doesn't respond to the *CATEGORISE* action notification within **1 hour**, this Beat task automatically moves the reel to **uncategorised** and sends a final confirmation push. It runs every 30 minutes via Celery Beat.

Constant	Value	Meaning
PENDING_CATEGORY_TIMEOUT_HOURS	1	Rows older than 1 hour get expired.
Beat schedule interval	30 min	Task runs twice per hour. Worst-case latency: 30 min after timeout.

```
def expire_pending_categories() -> dict:
    cutoff = datetime.now(timezone.utc) - timedelta(hours=1)

    # Single bulk UPDATE – the .eq('status', 'pending_category') guard ensures
    # rows already resolved by the user (via PATCH /category) are skipped.
    result = supabase.table('reels')
        .update({'status': 'uncategorised', 'suggested_categories': []})
        .eq('status', 'pending_category')
        .lt('updated_at', cutoff.isoformat())
        .execute()

    for row in (result.data or []):
        profile = supabase.table('profiles').select('fcm_token')
            .eq('id', row['user_id']).maybe_single().execute()
        fcm_token = (profile.data or {}).get('fcm_token')
        send_push_notification(
            fcm_token=fcm_token,
            title='Reel saved',
            body='Added to Uncategorised – you can move it anytime',
            data={'reel_id': row['id'], 'status': 'uncategorised'},
        )
```

Concurrency safety: Because the *UPDATE* filters on *status='pending_category'*, a user who tapped a button between the cutoff check and the bulk *UPDATE* will already have their row at *status='ready'* or *'uncategorised'*, so the update will skip it. No double notification can occur.

FILE

`backend/api/v1/reels.py`PATCH `/api/v1/reels/{reel_id}/category` — handles button taps and in-app category assignment.

This endpoint is called in two scenarios: (1) the iOS notification action handler (`ReelCategoryAPI.assign()`) fires it in the background when the user taps a button, and (2) `CategoriseReelView` fires the async variant when the user selects or creates a category in-app.

Path	Condition	DB update	Push sent
A — Pick category	<code>category_name</code> is a string	<code>category_id</code> set, <code>status=ready</code> , <code>confidence=1.0</code> , <code>suggested_categories=[]</code>	'Reel categorised!' + category name
B — Skip	<code>category_name</code> is null	<code>status=uncategorised</code> , <code>suggested_categories=[]</code>	'Reel saved — Added to Uncategorised'
Auto-create	Category name not found in DB	New row inserted into categories; then Path A	Same as Path A

Guards and edge-case handling:

- **404** — reel does not exist or does not belong to this user.
- **409 Conflict** — reel is already resolved (`status ≠ pending_category`). Prevents double-categorisation.
- **422** — empty `category_name` string after stripping whitespace.
- **Case insensitivity** — name is title-cased on input so 'travel vlogs' → 'Travel Vlogs' and matches existing categories case-insensitively.

10A · AppDelegate.didReceive(_:) — action handler

FILE

frontend/ReelMindApp.swift

UNUserNotificationCenterDelegate: routes each action button tap to the correct API call.

```
func userNotificationCenter(_ center: UNUserNotificationCenter,
    didReceive response: UNNotificationResponse, ...) {
    let userInfo = response.notification.request.content.userInfo
    guard let reelId = userInfo['reel_id'] as? String else { return }
    let suggestions = Self.parseSuggestions(from: userInfo) // decode JSON

    switch response.actionIdentifier {
    case 'CAT_0' where suggestions.count > 0:
        ReelCategoryAPI.assign(reelId: reelId, categoryName: suggestions[0])
    case 'CAT_1' where suggestions.count > 1:
        ReelCategoryAPI.assign(reelId: reelId, categoryName: suggestions[1])
    case 'UNCATEGORISED':
        ReelCategoryAPI.assign(reelId: reelId, categoryName: nil)
    case 'CHOOSE_IN_APP':
        NotificationCenter.default.post(name: .categoriseReel,
            object: nil, userInfo: ['reel_id': reelId, 'suggestions': suggestions])
    default: break
    }
}
```

parseSuggestions(from:) decodes the JSON string stored in the push data.suggestions field back into a [String] array, making it available for the button tap handlers.

10B · ReelCategoryAPI.swift — background PATCH

FILE

frontend/ReelCategoryAPI.swift

Fire-and-forget assign() + async assignAsync(). Both PATCH /api/v1/reels/{id}/category.

Two method variants serve different call sites:

- **assign(reelId:categoryName:)** — called from the notification action handler. Fire-and-forget URLSession.dataTask. Errors logged to console only.
- **assignAsync(reelId:categoryName:)** — called from CategoriseReelView. Returns async, throws on network error or non-2xx HTTP. Enables showing an error alert in the UI.

Both variants read the Supabase JWT from AppConfig.appGroupID App Group UserDefaults and send it as a Bearer header. If no auth token is present, the call is silently skipped (the share extension context may not have a session).

10C · CategoriseReelView.swift — in-app category picker

FILE

frontend/CategoriseReelView.swift

SwiftUI sheet opened when user taps 'Choose / Create Category'. Shows thumbnail, suggestions, all categories.

Shown when NotificationCenter.post(.categoriseReel) fires (from CHOOSE_IN_APP button) or when RootView routes to it. Features:

- Fetches reel thumbnail_url and caption from Supabase directly (using anon key + RLS).
- Shows AI suggestions as tappable chips at the top.
- Lists all user categories from the categories table.
- Allows creating a new category inline — passes the new name to assignAsync which auto-creates it on the backend.
- Toolbar 'Skip' button calls assign(nil) → moves to Uncategorized.
- Shows an error alert if the network call fails.

10D · Notification.Name.categoriseReel — internal routing

Defined in ReelMindApp.swift as an extension on Notification.Name. Posted by AppDelegate when CHOOSE_IN_APP is tapped; observed by RootView to present CategoriseReelView as a sheet with the reel_id and suggestions from the notification payload.

```
extension Notification.Name {
    static let categoriseReel = Notification.Name('categoriseReel')
}

// RootView listens:
.receive(NotificationCenter.default.publisher(for: .categoriseReel)) { notif in
    if let reelId = notif.userInfo?['reel_id'] as? String {
        categoriseTarget = CategoriseTarget(reelId: reelId, suggestions: suggestions)
    }
}
```

End-to-End FCM Flow — 14-Step Table

The table below traces a single reel from first launch through categorisation. Each row is one discrete step — the layer column is colour-coded to match the architecture diagram in Section 1.

#	Layer	Action	Outcome
1	iOS (Onboarding)	User taps 'Grant Access' in OnboardingPermissionsView	UNUserNotificationCenter.requestAuthorization() called
2	iOS (System)	System permission dialog → Granted	UIApplication.registerForRemoteNotifications() called
3	iOS (AppDelegate)	didRegisterForRemoteNotificationsWithDeviceToken	APNs token forwarded to Messaging.messaging().apnsToken
4	Firebase SDK (iOS)	SDK exchanges APNs token for FCM token	messaging(_:didReceiveRegistrationToken:) fires
5	iOS (AppDelegate)	FCM token received	Cached to UserDefaults('fcmToken'); if logged in → ProfileAPI.uploadFCMToken()
6	iOS (AuthSession)	User logs in or session restored	syncToken() calls ProfileAPI.uploadFCMToken(cached FCM token)
7	iOS → Backend	PATCH /api/v1/profiles/fcm-token {fcm_token: '...'}	profiles row updated with fcm_token + fcm_token_updated_at
8	Backend (Celery)	process_reel task runs pipeline Steps 15–20	Fetches fcm_token from profiles table at task start
9 A	Backend (Step 19)	Confidence ≥ 0.70 → auto-assign	Reel status → ready; send_push_notification('Reel saved!', 'Categorised as X')
9 B	Backend (Step 19)	Confidence < 0.70 → pending_category	Reel status → pending_category; push with category_id='CATEGORISE' + suggestions
9 C	Backend (Step 17)	NoSignalError (no transcript, caption, hashtags)	Reel status → uncategorised; push 'We couldn't categorise it'
1 0	Backend (notifier)	send_push_notification() via Firebase Admin SDK	_get_firebase_app() lazy init; messaging.send(_Message) dispatched
1 1	FCM Server (Google)	Message delivered to device via APNs	iOS receives push notification
1 2 A	iOS (Notification)	User taps CAT_0 or CAT_1 button	ReelCategoryAPI.assign(reelId, suggestions[0 or 1])
1 2 B	iOS (Notification)	User taps 'Uncategorised' button	ReelCategoryAPI.assign(reelId, nil)

#	Layer	Action	Outcome
1 2 C	iOS (Notification)	User taps 'Choose / Create Category' (.foreground)	NotificationCenter.post(.categoriseReel); RootView opens CategoriseReelView
1 3	iOS → Backend	PATCH /api/v1/reels/{reel_id}/category	Backend sets category_id, status=ready; confirmation push sent back
1 4	Backend (Beat)	expire_pending_categories() every 30 min	Rows pending >1 hr → uncategorised; push 'Added to Uncategorised' per user

Reel status transitions and their associated push notifications:

Status	Set by	Push notification sent
queued	POST /reels (API)	None
processing	tasks.py start	None
ready	tasks.py Step 19 (auto) OR PATCH /category	'Reel saved!' / 'Reel categorised!'
pending_category	tasks.py Step 19 (low conf)	'Help us categorise' with CATEGORISE action buttons
uncategorised	tasks.py Step 17 / Beat / PATCH nil	'Reel saved — Added to Uncategorised'
failed	tasks.py on non-retryable error	None